



PAYMENT RAILS AUDIT

A full, code-verified review of every payment path in Imotara.

Every gateway and billing surface was audited by reading the actual production code — not by trusting prior documentation. This report covers Razorpay, Stripe, Apple IAP, Google Play Billing, Imotara Connect's wallet/payout system, admin tooling, invoicing, and webhook security, plus the international-payments work that prompted the audit and the concrete plan to wire up Stripe for global payments.

● 12 areas audited

● 10 gaps found

● 2 already fixed

● Stripe wiring plan included

2 / 10Gaps already fixed & shipped
(v1.2.5)**2**

P1 — real risk, fix soon

5 + 3

P2 real gaps / P3 minor hygiene

20

Steps in the Stripe international-payments plan

Executive summary

The core payment infrastructure is solid: Razorpay (web), Apple IAP, and Google Play Billing all do real, correctly-verified transactions with proper webhook security. Ten concrete gaps were found across the mobile purchase paths, Connect's marketplace system, and admin tooling — two (the highest-severity ones) were fixed and shipped in the same release this audit produced. Separately, a session-long thread about accepting international payments concluded that Stripe — already built into the backend but never turned on — is the fastest path to global licensing and donations, pending a business-identity decision that needs a chartered accountant's sign-off before the Stripe account is opened.





WHAT WAS AUDITED, AND HOW

Every finding below came from reading the actual source code directly — Razorpay checkout/webhook/donations/corporate, Stripe checkout/webhook/cancellation, Apple IAP verification, Google Play Billing verification, the dead legacy Razorpay-Android path, Imotara Connect's wallet/recharge/payout/refund system, admin token tooling, invoice generation, and both gateways' webhook signature verification — not inferred from prior memory or documentation. Two focused sub-reviews ran across the mobile IAP code and the Connect/admin/invoicing code respectively, alongside direct checks of webhook signatures and live Vercel environment variables.

CONFIRMED SOLID — NO ACTION NEEDED

- ✓ **Razorpay web** (subscriptions, corporate, donations) — real orders, genuine HMAC-SHA256 signature verification with a timing-safe comparison; the previously-reported stuck-checkout-button bug is confirmed fixed on both the upgrade and donate pages.
- ✓ **Stripe backend** (checkout, webhook, cancellation) — fully built and correct; only needs an account, configuration, and frontend wiring, not more backend engineering (see the Stripe Wiring Plan on page 5).
- ✓ **Both webhook endpoints are genuinely secure** — Razorpay uses manual HMAC-SHA256 with a timing-safe comparison; Stripe uses its own official SDK's signature verification. Neither can be spoofed.
- ✓ **Apple IAP** — real server-to-server verification against Apple's App Store Server API, correct cryptographic signature verification, cross-checks the purchased product against Apple's own record, and self-heals if a prior grant attempt failed partway through.
- ✓ **Google Play Billing** — real server-to-server verification against the Android Publisher API, fails safely closed if credentials are missing. Two previously-reported bugs (a mislabeled payment source, and invoices showing the wrong currency) are confirmed fixed in the current code.
- ✓ **Imotara Connect's wallet** — recharge and top-up amounts are always recalculated server-side, never trusted from the client; the 80% consultant-earnings split and the minimum payout threshold are both correctly and consistently enforced, with solid protection against duplicate payout requests.
- ✓ **Admin token tooling** — supports both incremental and absolute balance edits, correctly blocks negative values, and logs every action to an audit trail.
- ✓ **Invoice generation** — a real, working generator that displays the correct currency for each payment gateway, not hardcoded to one currency.
- ✓ **Donation preset ordering** — confirmed clean and consistent on both web and mobile.
- ✓ **The old Razorpay-Android purchase path** — confirmed genuinely dead code with zero references anywhere; safe to remove whenever convenient, not a live risk today.





PRIORITY 1 — REAL RISK, ADDRESSED IMMEDIATELY

P1 **FIXED IN V1.2.5** **Apple IAP had no fail-closed protection**

If Apple's server credentials were ever absent or misconfigured, purchase verification was silently skipped entirely and the license was granted anyway — unlike the Google Play path, which correctly refuses the purchase in the same situation. Credentials are present in production today, so this was not actively exploitable, but it was a real, inconsistent latent risk. Now fails closed to match Google Play's behaviour.

[src/app/api/license/verify-apple-purchase/route.ts](#)

P1 **FIXED IN V1.2.5** **Purchases could go unacknowledged on two edge cases**

If a purchased product ID couldn't be mapped (iOS) or a purchase token was missing (Android), the code returned early without acknowledging the transaction — risking an automatic refund from Apple or Google within 3 days, or a purchase stuck permanently pending. Both edge cases now acknowledge the transaction before returning, matching the discipline already used on the normal success/cancel/error paths.

[imotara-mobile/src/components/imotara/UpgradeSheet.tsx](#)

PRIORITY 2 — REAL GAPS, LOWER URGENCY

P2 **The Android in-app-donation fix only reached one of two entry points**

A prior fix made the Settings screen's donate button open Razorpay checkout inside the app on Android. A second, separate donation entry point — the "Support Imotara" quick panel — still opens the external system browser instead. A genuine duplicate-surface miss that wasn't previously known.

[imotara-mobile/src/components/imotara/PlanSupportQuickPanel.tsx](#)

P2 **Google Play Billing lacks Apple's self-healing re-grant**

If the server crashes in the narrow window after recording a purchase but before granting the license, Apple's path automatically retries and corrects itself on the next check. Google Play's path has no equivalent protection — a retry in that exact window would report "already processed" without ever actually granting what was paid for.

[src/app/api/payments/google-play/verify/route.ts](#)

P2 **Android's "pending" purchase state isn't explicitly handled**

Deferred payment methods on Android produce a "pending" purchase state that isn't specifically handled in the purchase-error UI — only cancellation and "already owned" are. A user paying this way could be left in an unclear state with no explicit messaging.

[imotara-mobile/src/components/imotara/UpgradeSheet.tsx](#)



**P2 Mobile IAP invoices don't show the buyer's true currency**

Not a security issue, an accuracy one. Apple IAP invoices show "—" for the amount rather than a wrong figure (since the server can't know Apple's actual local-currency charge), which is at least honest but still uninformative. Google Play invoices now show a fixed USD amount — this fixed the previously-reported wrong-currency bug, but replaced it with a different fixed currency rather than the buyer's actual one.

[imotara-app invoice pipeline \(Apple & Google Play routes\)](#)

P2 DECISION NEEDED No real tier gate exists for the voice feature, despite the tutorial promising one

The in-app tutorial states plainly that the Free tier gets only the basic device voice while Plus and above unlock Imotara's neural voice. In the actual code, there is no tier or license check anywhere in the voice feature — every user, on every tier, gets the full neural voice by default. This is separate from the deliberate "everyone gets everything" soft-launch setting, which has nothing to do with voice specifically — this restriction appears to have been planned and documented, but never actually built. A decision is needed: correct the tutorial text to match reality, or build the gate to match what's already being promised and sold. This matters most once the soft-launch access setting is switched off.

[imotara-mobile/src/lib/tts/mobileTTS.ts](#) · [src/app/tutorial/page.tsx](#)

PRIORITY 3 — MINOR, COSMETIC OR HYGIENE**P3 A billing audit-trail write can fail silently**

On the admin license-update path, if the write to the audit-history table fails, it's only logged on the server — the admin using the panel still sees a success message. Low severity, but worth tightening specifically because this is the audit trail for billing actions.

[src/app/api/admin/licenses/\[userId\]/route.ts](#)

P3 Connect refunds are a manual process, by design

Not a bug — refund requests correctly create a ticket and notify support, to be actioned by a person within seven business days. Worth explicitly documenting as a manual process wherever it's surfaced, so nobody mistakes it for an automatic refund when reviewing the system later.

[Imotara Connect wallet refund-request flow](#)

P3 Dead code cleanup

The legacy Razorpay-based Android purchase function confirmed unreachable on page 2 can be deleted whenever convenient. No urgency, no risk in leaving it either way.

[imotara-mobile/src/components/imotara/UpgradeSheet.tsx](#)

Where this audit came from: a session-long thread that began with a simple question — how should Imotara accept payments from outside India? That exploration (a temporary PayPal bridge for donations, a look at making Razorpay itself international, and ultimately a decision to wire up Stripe) surfaced enough open questions about the wider payment system that a full audit made sense before building anything new on top of it. The international-payments plan is on the next page.





WHY STRIPE, AND NOT RAZORPAY-INTERNATIONAL OR A NEW PAYPAL ACCOUNT

Three options were compared for accepting payments from outside India: asking Razorpay to lift its existing international-payment block, opening a dedicated new PayPal Business account, or turning on Stripe. **Stripe won clearly** — its checkout, webhook, license-granting, invoicing, and cancellation logic are already fully built and sitting unused in the codebase, while the other two options each depend on a third party's approval timeline or need real integration work built from nothing.

THE PLAN — SIX PHASES

- 1 Business setup.** Resolve which legal identity the Stripe account should be opened under, then complete Stripe's own business verification. This is the one step blocked on a decision, not engineering — see the callout below.
- 2 Stripe Dashboard configuration.** Create the 8 price objects matching Imotara's existing product catalogue (4 subscription tiers, 4 token packs), and a webhook pointed at the existing, already-correct webhook endpoint.
- 3 Environment variables.** Add the Stripe secret key, webhook secret, publishable key, and all 8 price IDs to production, then redeploy.
- 4 Frontend — subscriptions & corporate.** The only real remaining engineering gap: add an actual button/flow that calls the already-built Stripe checkout route, on both the individual upgrade page and the corporate pricing page.
- 5 Donations via Stripe.** The other real gap — donations currently have no Stripe path at all, only Razorpay in Indian rupees. Needs a new route mirroring the existing donation flow, plus a currency picker for international donors.
- 6 Testing.** A full test-mode run across every purchase type, then one small real transaction in live mode before considering it done — the same rigor recommended for Google Play Billing's still-unverified real-money path.

What this plan deliberately excludes: mobile app-store purchases (Apple/Google already require and correctly handle their own in-app purchase systems — Stripe cannot and should not replace them there), and Imotara Connect's international counsellor payouts, which is a fundamentally different and larger problem. Paying a global network of consultants requires real multi-country payout infrastructure — a distinct product from a simple checkout, needing its own dedicated engineering phase once Connect is ready to onboard counsellors outside India. It is not live with international counsellors today, so there is no urgency to build this ahead of need.

The one open decision blocking Phase 1: which legal identity the new Stripe account (and ideally Razorpay's existing account) should be registered under. Imotara operates as a partnership firm with its own distinct PAN, but the payment processor accounts currently in use may not consistently match that legal entity. This is a genuine partnership-law and tax-structuring question — involving PAN registration, foreign-remittance purpose codes, and income-attribution rules — and needs a chartered accountant's confirmation before the new Stripe account is opened, not an engineering decision to make unilaterally.





Payment Rails Audit fixes

- ☒ P1 — ~~Make Apple IAP verification fail closed if credentials go missing~~
- ☒ P1 — ~~Acknowledge purchase transactions on the two edge case early returns~~
- ☐ P2 — Fix the second Android donation entry point to open checkout in-app, matching the first
- ☐ P2 — Add the same self-healing re-grant protection to Google Play Billing that Apple's path already has
- ☐ P2 — Explicitly handle Android's "pending" purchase state in the UI
- ☐ P2 — Improve currency accuracy on Apple/Google Play invoices to reflect the buyer's true purchase currency
- ☐ P2 — Decide and build the real voice-feature tier gate (or correct the tutorial to match current behaviour)
- ☐ P3 — Make audit-trail write failures surface rather than fail silently
- ☐ P3 — Explicitly document the Connect refund process as manual, wherever it's shown to users or admins
- ☐ P3 — Delete the confirmed-dead legacy Android purchase code

Stripe international-payments wiring

- ☐ **Phase 1** — Resolve the business-identity question with a chartered accountant, then open the Stripe account and complete verification
- ☐ **Phase 2** — Create the 8 price objects and the webhook endpoint in Stripe's dashboard
- ☐ **Phase 3** — Add all Stripe environment variables to production and redeploy
- ☐ **Phase 4** — Add the Stripe checkout trigger to the upgrade and corporate-pricing pages
- ☐ **Phase 5** — Build the Stripe donation flow, including a currency picker for international donors
- ☐ **Phase 6** — Full test-mode verification across every purchase type, then one real live-mode transaction

Status as of this report: the two highest-severity findings are already fixed and were shipped in production release v1.2.5 (iOS build 105, Android version code 105) on both platforms. Everything else above is scoped and ready to schedule for a future release cycle — none of it is blocking or urgent on its own.

